Open Source Architecture

Infraestrutura de Ciência de Dados e Big Data

Arquitetura Moderna com Ferramentas Open-Source

Professor: Benir Falcão Pistille

Navegação

Agenda

01 Introdução e Visão Geral

Conceitos fundamentais de Data Platform moderna e necessidades do setor financeiro

02 Ferramentas Open-Source

Stack completo: Spark, Kafka, Flink, MinIO, PostgreSQL, Airflow e Python

03 Arquitetura da Solução

Visão por camadas: Ingestão, Streaming, Armazenamento, Processamento e Consumo

04 Etapas de Implementação

Metodologia em 4 fases: Base, Ingestão, Processamento e Análise

05 Tecnologias Emergentes

Lakehouse, Data Mesh, Vector DBs, MLOps e Observabilidade 2024-2025

06 Camada de IA e MLOps

Frameworks ML, plataformas MLOps, Vector Databases e integração com Big Data

Visão Geral

Data Platform Moderna

Objetivo

Construir uma Data Platform robusta e escalável utilizando exclusivamente ferramentas open-source e gratuitas , focada nas necessidades específicas do setor financeiro.

> "O ambiente proposto é capaz de lidar com o volume massivo de transações, a velocidade exigida para análises em tempo real e a variedade de dados típicas do mercado financeiro."

Metodologia

Implementação dividida em etapas claras, culminando em um ambiente de desenvolvimento e produção totalmente funcional e reproduzível através do uso de contêineres Docker.

Desafios do Setor Financeiro

Volume Massivo
Milhões de transações diárias

Alta Velocidade
Análises em tempo real (fraudes)

Variedade de Dados
Estruturados e não estruturados

Benefícios da Abordagem

- Custo Zero Licenças
- Escalabilidade Total
- Comunidade Ativa
- Sem Vendor Lock-in

Stack Tecnológico

Ferramentas Open-Source Selecionadas

Escolha guiada por alta performance, baixa latência e ecossistema completo

Apache Spark
Processamento Distribuído

Framework unificado para processamento de dados em larga escala (batch e near real-time)

Casos: ETL, ML, análise de risco

Apache Flink
Processamento em Tempo Real

Framework streaming-first com baixa latência e estado consistente (stateful)

Casos: Detecção de fraudes, risco

Apache Kafka
Mensageria/Ingestão

Plataforma de streaming de eventos distribuída, alta vazão e tolerante a falhas

Casos: Transações, logs, cotações

MinIO
Data Lake

Armazenamento S3-compatible para dados brutos e processados

PostgreSQL
Data Warehouse

Banco relacional robusto para BI e relatórios estruturados

Apache Airflow
Orquestração

Gestão de workflows complexos e pipelines de dados

Python + ML
Análise e ML

Pandas, Scikit-learn, TensorFlow, PyTorch

Arquitetura

Visão da Arquitetura por Camadas

1 **Camada de Ingestão**
Apache Kafka recebe dados de transações, logs e dados de mercado em tempo real

2 **Camada de Streaming**
Apache Flink processa fluxo contínuo para análises de baixa latência

3 **Camada de Armazenamento**
MinIO atua como Data Lake central para dados brutos e processados

4 **Camada de Processamento**
Apache Spark executa transformações, ETL e treinamento de ML

5 **Camada de Consumo**
PostgreSQL oferece dados estruturados para BI e relatórios

Orquestração

Apache Airflow gerencia e monitora os pipelines de dados que movem e transformam dados entre todas as camadas da arquitetura.

Funcionalidades:

- ✓ Agendamento de jobs
- ✓ Monitoramento visual
- ✓ Gerenciamento de dependências
- ✓ Alertas automáticos

Princípios

- Modularidade
- Escalabilidade
- Resiliência
- Performance

Caso de Uso

Fluxo de Dados: Detecção de Fraude

Arquitetura permite fluxo de dados ponta a ponta em tempo real

ETAPA 1

Ingestão

Transações financeiras são geradas e enviadas ao Apache Kafka em tempo real, garantindo alta vazão e tolerância a falhas.

ETAPA 2

Streaming

O Apache Flink consome o fluxo, aplicando regras de negócio e modelos de ML de baixa latência para identificar padrões de fraude.

ETAPA 3

Processamento Batch

O Apache Spark, orquestrado pelo Airflow, processa o histórico de transações no MinIO para retreinar o modelo de detecção diariamente.

ETAPA 4

Consumo

Dados agregados sobre volume de fraudes e KPIs são carregados no PostgreSQL para dashboards de BI.

Métricas de Performance

Latência de Detecção	< 100ms
Throughput Kafka	2M+ msg/s
Precisão do Modelo	99.5%
Disponibilidade	99.99%

Benefícios

- Detecção em tempo real
- Modelos continuamente atualizados
- Visão holística dos dados
- Escalabilidade automática

Metodologia

Etapas de Implementação

Projeto dividido em 4 etapas principais para implementação gradual

1 Base e Armazenamento

Fundamentos da infraestrutura

Objetivo

Estabelecer o ambiente operacional e o Data Lake

Ferramentas

Docker | MinIO | PostgreSQL

2 Ingestão e Streaming

Fluxo de dados em tempo real

Objetivo

Configurar plataforma de mensageria e processamento de eventos

Ferramentas

Apache Kafka | Apache Flink

3 Processamento e Orquestração

Transformação e gestão de dados

Objetivo

Implementar processamento em lote e gestão dos pipelines

Ferramentas

Apache Spark | Apache Airflow

4 Análise e Consumo

Ciência de Dados e BI

Objetivo

Configurar ambiente de desenvolvimento e ferramentas de análise

Ferramentas

Python | Jupyter | ML Libs

Inovações 2024-2025

Tecnologias Emergentes

Tendências que complementam e expandem a arquitetura de dados moderna

Lakehouse Architecture

Combinação de Data Lake e Data Warehouse com **transações ACID**, evolução de schema e otimizações de performance.

Open Table Formats

Apache Iceberg | Delta Lake | Apache Paimon

Benefício: Streaming lakehouse com LSM-tree

Data Mesh

Arquitetura **descentralizada** onde domínios de negócio possuem seus dados como produtos, com governança federada.

Princípios Fundamentais

- Domain-oriented ownership
- Data as a Product
- Self-serve infrastructure
- Federated governance

Mercado: $1.2B → $2.5B até 2028

Vector Databases

Bancos especializados para **embeddings e RAG**, essenciais para aplicações de IA e busca semântica.

Principais Soluções

Milvus | Qdrant | Weaviate | pgvector

Uso: RAG, similaridade semântica

MLOps Tools

Plataformas para **gerenciamento completo** do ciclo de vida de ML: experimentação, deployment e monitoramento.

Ferramentas Open Source

MLflow | Kubeflow | DVC

Funcionalidades: Tracking, Registry, Serving

Data Observability

Monitoramento de **qualidade, lineage e health** dos dados em tempo real, com detecção automática de anomalias.

Stack de Observabilidade

OpenTelemetry | Grafana | Prometheus

76% das empresas adotam OTel

Edge Computing

Processamento de dados **próximo à fonte**, reduzindo latência e bandwidth para IoT e aplicações em tempo real.

Casos de Uso

- IoT e sensores industriais
- Veículos autônomos
- Smart cities

75% dos dados serão processados no edge

Inteligência Artificial

Camada de IA e MLOps

Integração completa de Machine Learning com a infraestrutura de Big Data

Frameworks de ML

TensorFlow — Google
Deep learning, redes neurais, produção em escala

PyTorch — Meta
Pesquisa, flexibilidade, dynamic computation

Scikit-learn — Classic
ML tradicional, classificação, regressão, clustering

Plataformas MLOps

MLflow — Databricks
Gerenciamento completo do ciclo de vida de ML
Tracking | Projects | Models | Registry

Kubeflow — Google
ML workflows nativos em Kubernetes
Pipelines | Katib | KFServe | Notebooks

Bibliotecas Python

Pandas
Manipulação

NumPy
Computação

Matplotlib
Visualização

PySpark MLlib
Distributed

Vector Databases

Essenciais para RAG (Retrieval Augmented Generation) e busca semântica em aplicações de IA.

- Milvus — Cloud-native
- Qdrant — Rust-based
- Weaviate — Knowledge Graph

Implementação

Configuração com Docker Compose

Reprodutibilidade e facilidade de uso com containers

Pré-requisitos

Docker Engine
Container runtime

Docker Compose
Orquestração multi-container

Python 3.x
Ambiente de desenvolvimento

Serviços Configurados

Zookeeper + Kafka — Portas: 2181, 9092
Mensageria e streaming de eventos

MinIO — Portas: 9000, 9001
Data Lake S3-compatible

PostgreSQL — Porta: 5432
Data Warehouse analítico

Apache Airflow — Porta: 8080
Orquestração de workflows

Estrutura de Diretórios

```
bigdata-finance-project/
├── docker-compose.yml
├── data_lake/# MinIO
├── postgres_data/# PostgreSQL
├── airflow_dags/# Pipelines
└── airflow_logs/# Logs
```

Inicialização

```
# Iniciar todos os serviços
docker compose up -d
```

MinIO Console
localhost:9001

Airflow Web
localhost:8080

Desenvolvimento

Ambiente de Ciência de Dados

Configuração Python com PySpark e integrações

Instalação de Bibliotecas

```
# Instalar dependências
pip install pyspark \
        kafka-python \
        minio \
        psycopg2-binary \
        pandas \
        jupyterlab
```

Conexões e Testes

PySpark + MinIO
Configuração S3-compatible para acesso ao Data Lake

Kafka Producer
Envio de mensagens de teste para tópicos

PostgreSQL
Leitura e escrita de dados estruturados

Exemplo: Pipeline Airflow

```
# etl_pipeline.py
from
airflow
import
DAG
from
airflow.operators.bash
import
BashOperator
from
airflow.utils.dates
import
days_ago
with
DAG(
    dag_id=
'finance_etl_pipeline'
,
    start_date=days_ago(
1
),
    schedule_interval=
'@daily'
,
    catchup=
False
,
)
as
dag:
    spark_task = BashOperator(
        task_id=
'run_pyspark_job'
,
        bash_command=
'spark-submit ...'
    )
    quality_check = BashOperator(
        task_id=
'data_quality_check'
,
        bash_command=
'echo "Quality OK"'
    )
    spark_task >> quality_check
```

Conclusão da Aula

Benefícios da Infraestrutura

Flexibilidade Total
Ferramentas open-source sem vendor lock-in

Escalabilidade Horizontal
Cresce conforme demanda do negócio

Baixa Latência
Processamento em tempo real para detecção de fraudes

Reprodutibilidade
Ambiente consistente com Docker Compose

Stack Tecnológico

- Spark
- Flink
- Kafka
- MinIO
- PostgreSQL
- Airflow
- Python
- ML/AI

Recomendações

1 Implementação Gradual
Seguir as 4 etapas propostas, validando cada camada antes de prosseguir

2 Testes em Ambiente Dev
Validar todos os componentes antes de migrar para produção

3 Monitoramento Contínuo
Implementar observabilidade desde o início do projeto

4 Expansão para MLOps
Adicionar MLflow ou Kubeflow para gerenciamento de modelos

100%
Open Source
Sem custos de licenciamento